

Core Java vs Advanced Java

- Core Java focuses on OOPs, Collections, Multithreading, and I/O.
- Advanced Java includes Servlets, JSP, JDBC, Spring, Hibernate.
- Core Java is used for standalone apps; Advanced Java for web/enterprise apps.

Web Applications (Client–Server Architecture)

- Client (browser) sends HTTP request → Server processes → Response sent back.
- Server hosts Servlets/JSP which interact with the database.
- Tomcat or GlassFish act as web servers handling requests.

MVC Architecture

- Model – handles business logic and database (Java + JDBC/Hibernate).
- View – user interface (JSP/HTML).
- Controller – handles requests (Servlets).
- Separates logic, UI, and control for clean code.

JDBC

- Java API to connect and execute SQL on databases.
- Steps: Load driver → Establish connection → Create statement → Execute query → Close connection.
- Uses Statement, PreparedStatement, and ResultSet classes.

Servlets

- Java classes that handle HTTP requests and responses on the server.
- Lifecycle: init() → service() → destroy().
- ServletConfig gives config info; ServletContext gives global info.

JSP (JavaServer Pages)

- Used for creating dynamic web content easily.
- JSP lifecycle: Translation → Compilation → Initialization → Execution → Destruction.
- JSP Elements: Scriptlets <% %>, Declarations <%! %>, Expressions <%= %>.
- Implicit objects: request, response, session, application, out.

Session Management

- Cookies – stored in client browser.
- URL Rewriting – session info added to URL.
- Hidden Form Fields – data stored in form fields.
- HttpSession – server-side session management.

Spring Framework

- Lightweight framework for building enterprise Java apps.
- Features: Inversion of Control (IoC) and Dependency Injection (DI).
- Modules: Spring Core, Beans, Context, DAO, MVC.

Hibernate ORM

- ORM maps Java objects to database tables.
- Components: SessionFactory, Session, Transaction, Query.
- Advantages: Less SQL, database independent, supports caching.
- Mapping via XML or Annotations.
- Connects DB using hibernate.cfg.xml and SessionFactory.

JDBC vs Hibernate

- JDBC requires manual SQL and mapping; Hibernate automates both.
- Hibernate supports caching and ORM; JDBC is low-level API.

Spring + Hibernate Integration

- Spring manages app logic and dependency injection.
- Hibernate manages data persistence and ORM.
- Together provide clean separation, easier transactions, and scalability.

Mini Project Example

- Student Management System using JSP (View), Servlet (Controller), JDBC (Model).
- Flow: JSP form → Servlet → JDBC → Database → JSP output.
- Demonstrates MVC and CRUD operations.